

Étude de faisabilité — Système d'analyse vidéo (board-to-FEN)

Auteur : Arnaud Pinsun

1. Contexte & Problématique

Le POC actuel propose un agent d'apprentissage des ouvertures d'échecs. Lorsqu'une position est analysée, l'agent recommande des vidéos YouTube via une recherche **textuelle** (ex. "Sicilienne chess opening tutorial").

« La simple requête textuelle pour obtenir la bonne vidéo YouTube est limitante. On risque de proposer une vidéo de 45 minutes sur la Sicilienne alors que l'utilisateur a juste besoin de l'explication du coup en cours. »

Objectif : pouvoir retrouver, dans un catalogue de vidéos, le **moment exact** où une position donnée apparaît, et renvoyer le lien avec le **timestamp précis**.

2. Le système proposé

Principe général

Un service qui, étant donné une position FEN, renvoie la vidéo et le timestamp correspondant à cette position.

Le pipeline d'ingestion

Vidéo YouTube

| 1. Téléchargement



Fichier vidéo

| 2. Extraction frame par frame (ffmpeg)



Frames (images)

| 3. Détection de l'échiquier (classifieur / segmentation)

| — la frame contient-elle un échiquier ? si oui, l'extraire —



Image d'échiquier cadrée

| 4. Conversion board → FEN (modèle de vision, ex. fenify-3D)



Position FEN

| 5. Validation + correction



Indexation : { fen → [(video, timestamp)] }

Le point clé : la correction d'erreurs

Le modèle de reconnaissance commet des erreurs (en moyenne ~3 cases fausses sur 64). Deux mécanismes de correction :

1. **Validation par les règles d'échecs** — une position illégale (deux rois blancs, pion sur la première rangée...) est rejetée ou corrigée.
2. **Cohérence temporelle** — pour une frame f , on compare avec les frames $f-1$ et $f+1$. On suppose que les pièces ne « se téléportent » pas : si une case change d'état de façon isolée et incohérente entre frames voisines, c'est probablement une erreur du modèle, corrigeable par vote majoritaire sur une fenêtre de frames.

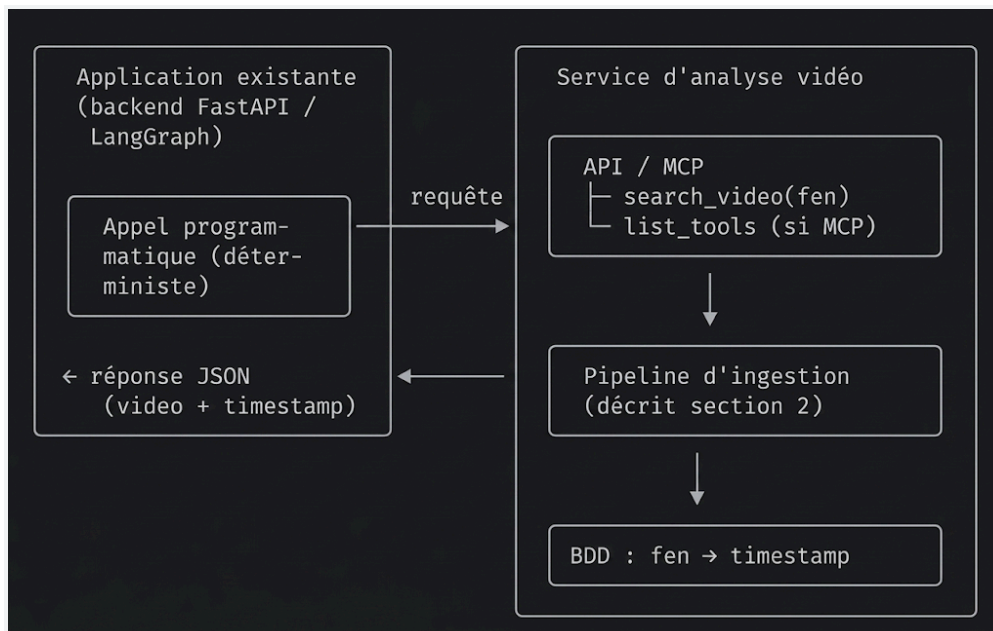
Note : Ce second mécanisme est particulièrement pertinent : dans une vidéo, la position est généralement **stable** pendant plusieurs frames. On peut donc exploiter la redondance temporelle pour fiabiliser la détection, bien au-delà de la précision brute du modèle.

Stockage

Chaque vidéo analysée produit un index : $\{ fen \rightarrow [(video_id, timestamp)] \}$, stocké dans une base de données. La position FEN sert de **clé de recherche**.

3. Architecture technique & protocole MCP

Le schéma d'architecture



Le protocole MCP — et pourquoi il n'est pas nécessaire ici

Le brief suggère d'exposer ce service via un serveur **MCP (Model Context Protocol)**. Après analyse, nous concluons que **MCP n'apporte pas de valeur dans notre contexte**.

Rappel : MCP est un protocole standardisé (JSON-RPC) qui permet à un LLM de **découvrir et choisir dynamiquement** des outils. Son intérêt apparaît quand :

- plusieurs applications IA différentes doivent partager les mêmes outils,
- le LLM doit sélectionner un outil parmi plusieurs selon le contexte.

Or, dans notre architecture :

- la position FEN est **déjà connue** (elle circule dans l'état du graphe LangGraph),
- l'appel au service est **déterministe** (une position → une recherche),
- aucun choix d'outil n'est nécessaire.

→ Un simple appel programmatique (requête HTTP à un microservice) suffit. Ajouter une couche MCP n'apporterait que de la complexité, sans bénéfice.

Néanmoins, si l'on devait l'exposer en MCP...

Le serveur serait minimal :

Outils exposés :

```
├─ list_tools()      (obligatoire)
└─ search_video(fen) → [{ video_url, timestamp, title }]
```

Un seul outil métier. C'est un bon indicateur du décalage : **un protocole de découverte d'outils pour exposer un seul outil**.

Le cas où MCP reprendrait du sens

Si la FFE souhaitait que cette brique vidéo soit **réutilisable par d'autres outils IA** (Claude Desktop, un autre agent, un assistant d'entraîneur), alors MCP deviendrait pertinent : le service serait branchable partout sans recoder. Pour le POC, ce n'est pas le besoin.

Recommandation : pour le POC, exposer le service comme une **API REST simple** (FastAPI), et réserver MCP à une évolution ultérieure si la réutilisabilité multi-agents devient un besoin.

4. Bénéfices attendus

Précision — le bon timestamp, pas la bonne vidéo

Actuellement, la recherche textuelle renvoie des vidéos **entières**. Un jeune joueur qui étudie la variante Najdorf de la Sicilienne recevrait une vidéo de 45 minutes sur la Sicilienne en général, alors que le passage qui l'intéresse n'occupe que 3 minutes. En indexant chaque frame par sa position FEN, le système localise le **moment exact** où la position apparaît. L'utilisateur est redirigé vers le bon timestamp : 40 minutes de recherche manuelle économisées.

Expérience utilisateur

Pour un jeune en apprentissage, dont l'attention est limitée, cette précision est déterminante. Il va droit au but, voit la position concrète à l'écran, et reste engagé. La vidéo devient un **support pédagogique ciblé** plutôt qu'un contenu à fouiller.

Scalabilité du catalogue

Une fois le pipeline en place, chaque nouvelle vidéo ajoutée au catalogue est automatiquement indexée par positions FEN. Le catalogue devient une **base interrogeable** : n'importe quelle position de n'importe quelle partie peut être retrouvée, sans effort humain de description ou de tagging.

Valeur pédagogique pour la FFE

Le système s'intègre naturellement à l'agent existant : quand un jeune joueur sort de la théorie, l'agent peut lui dire « *regarde cette vidéo, à 14:32, on voit exactement ta position et la réponse* ». C'est un levier d'apprentissage concret, aligné avec la mission de la FFE (accompagner les jeunes espoirs).

5. Limites techniques

La distinction fondamentale : plateau physique vs numérique

C'est le facteur qui conditionne toute la faisabilité technique. Les vidéos d'échecs se répartissent en deux catégories très différentes :

Type	Exemple	Difficulté de détection
Numérique (2D)	Streamers sur Lichess/chess.com	● Faible (~100 % de précision possible)
Physique (3D)	Tutoriels sur vrai plateau (angles, éclairage)	● Élevée

- **Pour le numérique**, des solutions existantes atteignent une précision quasi parfaite (ex. chess-scanner : 1000/1000 positions correctes sur des boards Lichess/chess.com).
- **Pour le physique**, c'est le vrai défi : il faut détecter l'échiquier sous différents angles et éclairages, puis reconnaître des pièces réelles, potentiellement occluses.

La détection d'échiquier

La méthode classique OpenCV (cv2.findChessboardCorners) **échoue dès qu'il y a des pièces sur le plateau** (elle attend un damier nu de calibration). Des alternatives existent :

- approches OpenCV dédiées (détection de lignes + contours, ex. chessboard-detection-opencv),
- modèles de segmentation/détection d'angles entraînés.

Cette étape est un **prérequis** : le modèle board→FEN suppose une image déjà cadrée sur l'échiquier.

La reconnaissance board → FEN (fenify-3D)

Le modèle envisagé (fenify-3D, EfficientNetV2_S) présente des caractéristiques à connaître :

Caractéristique	Valeur
Inférence	✓ CPU (léger, tourne sur matériel modeste)
Entraînement	✗ GPU V100 (~17 h) — mais on utilise les poids pré-entraînés
Précision	~95 % par case, soit ~3 cases fausses sur 64
Poids du modèle	⚠ non publiés dans le repo GitHub

Le dernier point est important : les poids pré-entraînés ne sont pas distribués avec le code. Il faudrait les obtenir auprès de l'auteur, ou ré-entraîner le modèle — ce qui change radicalement le coût.

Le post-traitement est indispensable

Avec ~3 cases fausses par frame, le FEN brut est souvent incorrect. Deux correctifs sont nécessaires :

1. **Validation par les règles d'échecs** (rejet des positions illégales),
2. **Cohérence temporelle** (vote majoritaire sur frames voisines — voir section 2).

Ce post-traitement est aussi important que le modèle lui-même : c'est lui qui rend la sortie exploitable.

L'invisibilité du trait : Suivi d'état (State Tracking) vs Réalité théorique

Un modèle de vision tel que **fenify-3D** présente une limite inhérente : il ne peut extraire que la disposition spatiale des pièces (le premier segment de la chaîne FEN). Les données immatérielles, telles que le trait (à qui le tour de jouer), les droits de roque ou les possibilités de prise en passant, sont purement invisibles sur une image fixe.

Pour pallier ce manque et reconstituer un FEN complet et valide, le pipeline d'ingestion devrait implémenter un algorithme de *State Tracking* (suivi d'état temporel). Le système devrait identifier la position de départ initiale (où le trait est connu), puis chronométrer l'apparition des nouvelles positions pour inverser logiquement le trait à chaque demi-coup détecté. Cette approche ajoute cependant une **complexité algorithmique et une fragilité importantes** au code : il faudrait alors gérer les coupures au montage de la vidéo, les sauts de chapitres, ou encore les explications où le présentateur revient en arrière sur l'échiquier.

Il convient toutefois de nuancer fortement cette limite technique au regard de notre périmètre métier : **l'apprentissage des ouvertures**.

Dans le cadre strict de la théorie des ouvertures, la configuration spatiale des pièces agit comme une empreinte digitale unique de la position. Il est virtuellement impossible d'atteindre une disposition exacte des pièces avec un

trait inversé, à moins de réaliser une séquence de coups aberrante impliquant une perte de "tempi" (temps) volontaire – un scénario qui ne figurera jamais dans une vidéo tutorielle de la FFE.

Par conséquent, utiliser uniquement le "Board FEN" (la partie visuelle du FEN) comme clé d'indexation et de recherche dans la base de données est une heuristique extrêmement robuste. Cela permet de s'affranchir d'un développement complexe et coûteux lié au suivi d'état vidéo, sans générer de faux positifs pour nos jeunes joueurs.

6. Limites business

Coût d'infrastructure vs valeur ajoutée

Le coût d'analyse GPU est faible (voir section 7), mais il faut le rapporter à la valeur : l'indexation ne vaut que si le catalogue est réellement utilisé. Pour un POC avec un petit volume de vidéos, l'investissement en développement doit rester proportionné.

Maintenance des modèles

Les modèles de vision évoluent et peuvent se dégrader sur de nouveaux types de plateaux ou de pièces (le modèle fenify-3D reconnaît mal certaines pièces selon le design). Une veille et un ré-entraînement ponctuel sont à prévoir.

Volume de vidéos

Le coût est linéaire en nombre de vidéos analysées. Le choix des vidéos à indexer (qui les sélectionne ? avec quels droits ?) est une question business autant que technique : analyser des vidéos YouTube soulève des questions de droit d'auteur à cadrer avec la FFE.

7. Étude de coûts

Hypothèses

- 100 vidéos initiales, 30 minutes chacune
- 1 frame extraite par seconde (≈ 1800 frames/vidéo)
- Modèle board→FEN **pré-entraîné** (pas de ré-entraînement)
- GPU cloud de type L4 (~ 1 €/h)

Build (CAPEX — une seule fois)

Poste	Estimation
Développement du pipeline (2-3 semaines ingénieur IA)	10-15 k€

Poste	Estimation
Sélection / évaluation des modèles	2-3 k€
Intégration + tests	3-5 k€
Total build	~15-25 k€

OPEX (mensuel, en fonctionnement)

Poste	Calcul	Estimation
Analyse GPU	100 vidéos × 1800 frames × ~75 ms ≈ 3,75 h GPU	~5-10 €
Stockage	~50 Go (vidéos + frames + vecteurs)	~2-3 €/mois
Maintenance	0,5 jour/mois	~1-1,5 k€/mois
Total OPEX		~1-1,5 k€/mois

Analyse

Le poste **dominant est le développement** (build), pas le calcul. Analyser 100 vidéos coûte quelques euros de GPU. Ce qui coûte cher, c'est la construction et la maintenance du pipeline.

Arbitrage : API cloud vs GPU réservé

Approche	Coût fixe	Coût variable	Quand
API cloud de vision (par frame)	~0	élevé à l'échelle	POC, faible volume
GPU réservé /	investissement initial	faible	Production, volume

Approche	Coût fixe	Coût variable	Quand
self-hosted			régulier

→ Pour le POC : API cloud. Pour la production FFE : GPU réservé.

Conclusion économique

Le système est **économiquement viable** : OPEX < 2 k€/mois pour 100 vidéos, avec un build initial d'environ 20 k€. Le coût dominant étant le temps humain, la réussite dépend plus de l'exécution du développement que de l'infrastructure.

8. Alternatives & Roadmap

Alternatives envisagées

Alternative	Description	Avantage	Inconvénient
Numérique d'abord	N'indexer que les vidéos à plateau 2D (Lichess/chess.com)	Précision ~100 %, pipeline plus simple	Ne couvre pas les vidéos physiques
Microservice simple (sans MCP)	Exposer le service en API REST FastAPI	Simple, suffisant pour le POC	Moins réutilisable
Métadonnées manuelles	Taguer les vidéos à la main (sans vision)	Zéro coût technique	Non scalable, subjectif
MCP complet	Serveur MCP réutilisable multi-agents	Portabilité	Complexité inutile à ce stade

Roadmap de développement

- Phase 1 — Plateau numérique
- Pipeline complet sur vidéos 2D, précision ~100 %
- ↓ (valeur immédiate, risque faible)
- Phase 2 — Plateau physique
- Détection d'échiquier + fenify-3D + post-traitement temporel

↓ (le vrai défi technique)

Phase 3 — Exposition MCP (si besoin)

Wrapper MCP sur le même service, sans modifier le cœur

↓ (uniquement si réutilisabilité multi-agents demandée)

Cette progression permet de livrer de la valeur rapidement, tout en gardant l'option MCP ouverte pour plus tard — le service restant identique, seul le protocole d'exposition changerait.

9. Risques & Conclusion

Risques

Risque	Type	Gravité	Mitigation
Poids de fenify-3D non publiés	Technique	● Haute	Obtenir les poids, ou utiliser un modèle alternatif
Précision insuffisante sur plateau physique	Technique	● Moyenne	Post-traitement temporel + commencer par le numérique
Dérive des modèles sur nouveaux designs	Technique	● Faible	Ré-entraînement ponctuel
Coût de développement sous-estimé	Business	● Moyenne	Roadmap en phases, MVP numérique d'abord
Droits d'auteur sur les vidéos analysées	Business	● Moyenne	Cadre juridique à clarifier avec la FFE

Conclusion

Le système d'analyse vidéo (frames → échiquier → FEN → timestamp) est **techniquement et économiquement faisable**. Le principal défi est la reconnaissance sur **plateau physique**, qui impose un post-traitement robuste (validation par règles + cohérence temporelle).

Recommandation finale :

1. Commencer par le **plateau numérique** (précision ~100 %, valeur immédiate).
2. Exposer le service en **API REST simple** — MCP n'est pas justifié pour un appel déterministe.
3. Traiter le plateau physique en **phase 2**, avec le post-traitement comme brique centrale.
4. Réserver **MCP** à une évolution ultérieure si la réutilisabilité multi-agents devient un besoin réel.